Internet Of Things (IOT) - Challenge 3 – Part2 - Exercise
Authors: Mostafa Moazenkakhki, Seyed Mohammad Tabatabei

EQ1) A LoRaWAN network in Europe (carrier frequency 868 MHz, bandwidth 125 kHz) is composed by one gateway and 50 sensor nodes. The sensor nodes transmit packet with payload size of L byte according to a Poisson process with intensity lambda = 1 packet / minute. Find the biggest LoRa SF for having a success rate of at least 70%. Hint: use https://www.thethingsnetwork.org/airtime-calculator to compute the airtime of a packet.

Solution:

Person code: 10869960

XY = 60

L = 3 + XY bytes = 36

Total transmission rate:

Each node sends 1 packet/minute = 1/60 packet/second

$$\lambda = 1\text{pkt/min} = \frac{1}{60}\text{pkt/sec}$$

50 sensor nodes

Total rate $= \boldsymbol{N} \times \boldsymbol{\lambda}$ = 50 × (1/60) = 0.833 packets/second

ALOHA success model formula:

$$DER = e^{(-2N \cdot T_{packet} \cdot \lambda)} \qquad (12)$$

N is the number of transmitters, Tpacket the packet airtime and lambda is the transmission rate of all nodes.

$$G = N \cdot \lambda \cdot t = 0.833 \cdot t$$

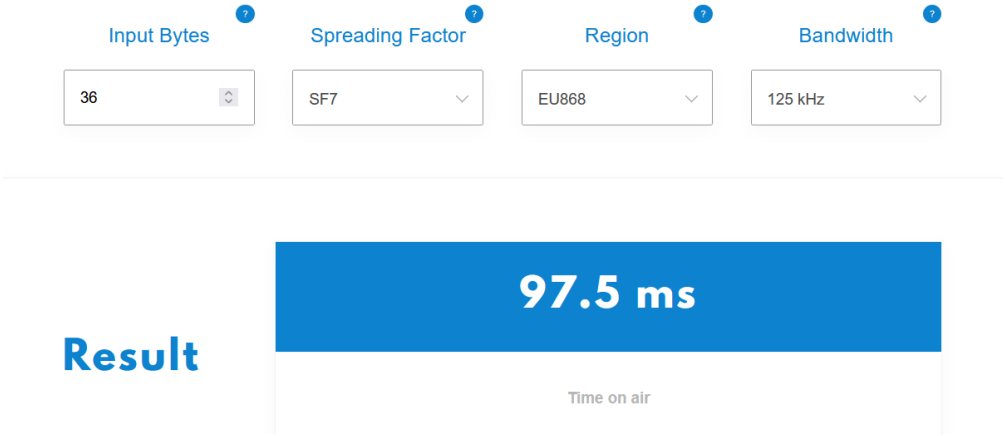
$$\text{Success Rate} = e^{-2G}$$

We want to:

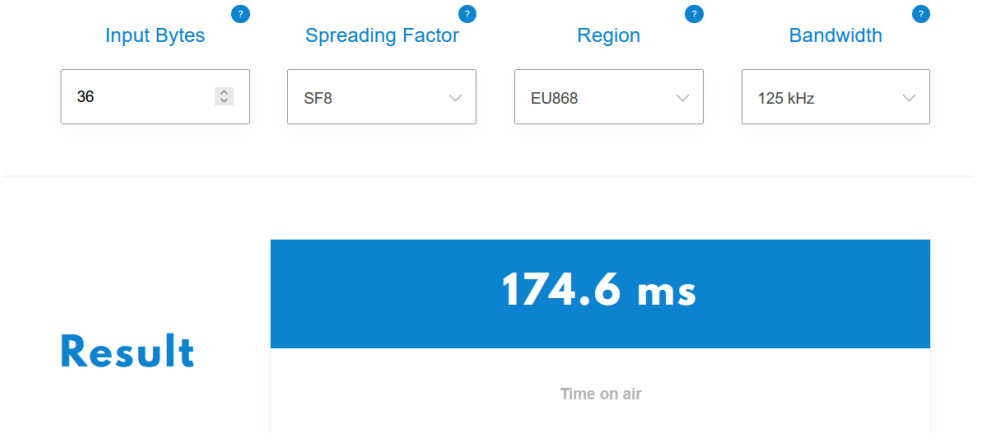
$$\text{Success Rate} \geq 0.70 \Rightarrow G \leq -\frac{\ln(0.7)}{2} \approx 0.178 \Rightarrow t \leq \frac{0.178}{0.833} \approx 0.214 \text{ second}$$

These are the time on airs of different spreading factors (SF7, SF8, SF9), computed with the things network:

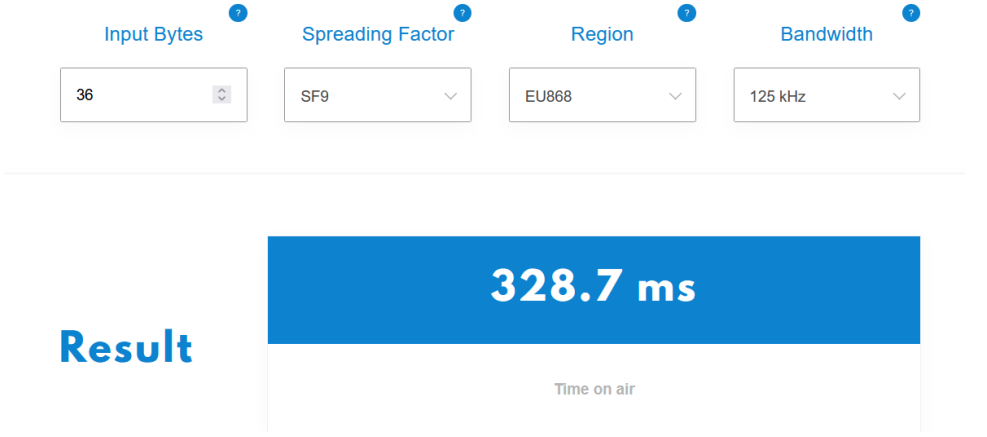
SF7:


Input Bytes
Spreading Factor
Region
Bandwidth
36
SF7
EU868
125 kHz
Result
97.5 ms
Time on air

SF8:


Input Bytes
Spreading Factor
Region
Bandwidth
36
SF8
EU868
125 kHz
Result
174.6 ms
Time on air

SF9:


Input Bytes
Spreading Factor
Region
Bandwidth
36
SF9
EU868
125 kHz
Result
328.7 ms
Time on air

SF	Airtime (ms)	≥70%?
7	**97.5**	**YES**
8	**246.3**	**YES**
9	451.6	NO

As seen in the results for getting success rate of at least 70%, the largest spreading factor that is allowed is **SF8**

Ex2:

An Arduino MKR WAN 1310 was purchased to build a system where temperature and humidity data would be read from a DHT22 sensor and sent wirelessly to ThingSpeak using LoRaWAN. A complete system diagram would be designed (in Node-RED) and the setup steps would be explained.

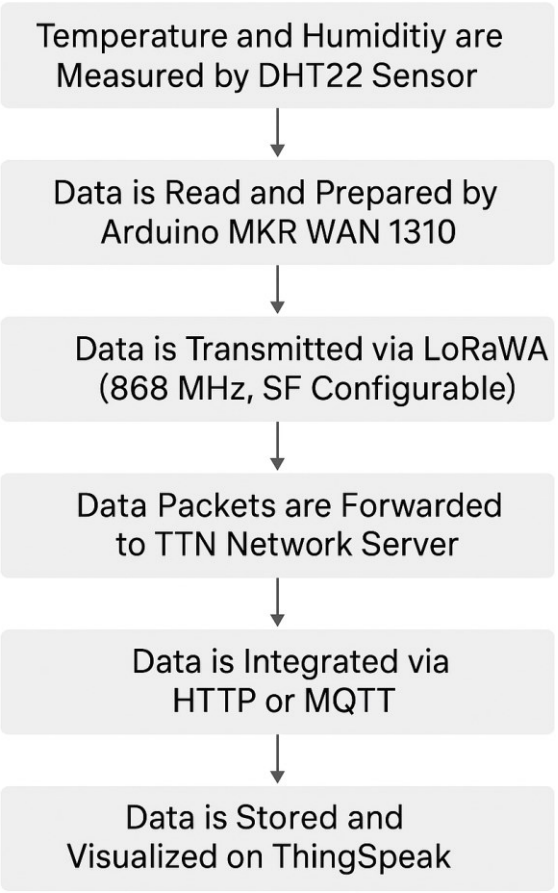
System Flow:

1. **DHT22 Sensor**
   - Temperature and humidity are measured.
   - The data is sent as a digital signal.
2. **Arduino MKR WAN 1310**
   - The sensor data is read.
   - The DHT sensor library is used to process the readings.
   - The LoRaWAN library is used to prepare and transmit the data wirelessly.
3. **LoRa Transmission**
   - The sensor data is transmitted over 868 MHz (the standard frequency in Europe).
   - The Spreading Factor (SF) can be adjusted to control data rate, range, and power usage.
4. **LoRa Gateway (e.g., The Things Network Gateway)**
   - The LoRa signal is received from the Arduino.
   - The data is forwarded to the Internet.
5. **The Things Network (TTN) Server**
   - The device is authenticated and the data is checked.
   - The data is prepared for integration through HTTP or MQTT.
6. **Integration (HTTP or MQTT)**
   - The data is forwarded to an external platform such as ThingSpeak.
   - Either HTTP POST requests or MQTT publishing is used.
7. **ThingSpeak**
   - The data is stored (e.g., Field 1 = temperature, Field 2 = humidity).
   - The data is displayed in real-time charts.
   - Optional alerts or actions can be set up based on the incoming data.

Node-RED Simulation:

Since the physical devices are not available, the system will be simulated in Node-RED based on this flow.

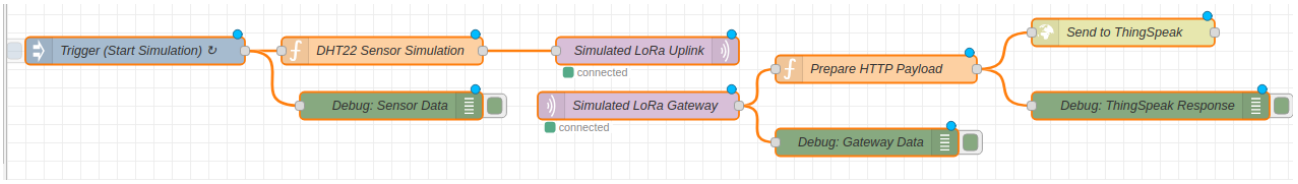
Here is a simple block diagram:



Node-RED Flow Report

Flow Description:

This flow simulates sending temperature and humidity sensor data through a LoRaWAN network and uploads it to ThingSpeak via HTTP. Since physical devices aren't used, everything is simulated inside Node-RED.



Here’s what each node does:

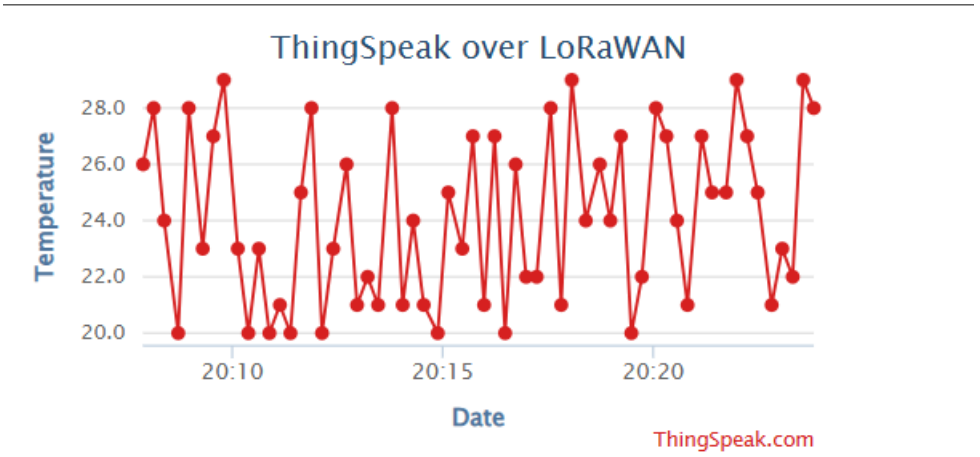
timestamp
Generates a trigger (timestamp) at regular intervals to start the data simulation process.
Simulated Sensor
Mimics the behavior of a real DHT22 sensor by producing random (but realistic) temperature and humidity values.
debug 2
Displays the simulated sensor data for verification and troubleshooting.

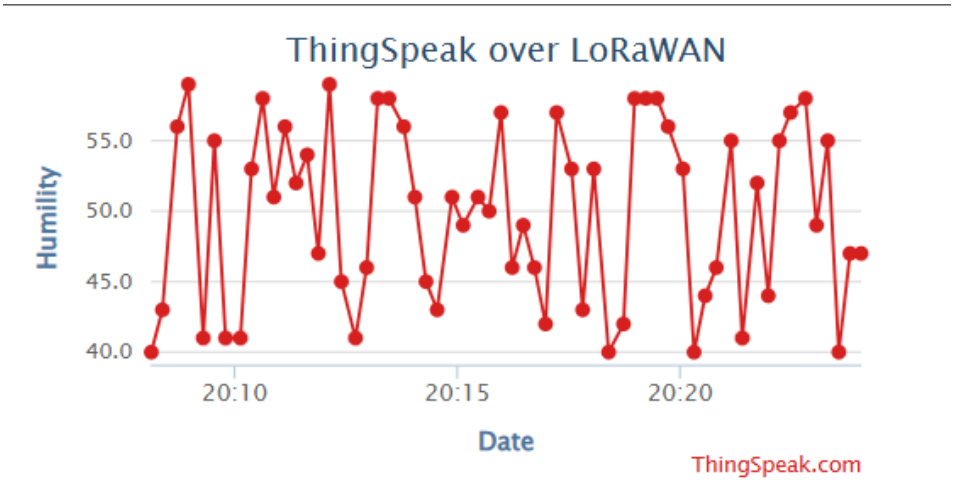
LoRa Transmission
Simulates the LoRaWAN transmission by packaging the sensor data into a format that would normally be transmitted wirelessly.
LoRa Gateway Receiver
Simulates the behavior of a LoRa gateway receiving the transmitted data and forwarding it to the network server.
debug 3
Displays the received data at the gateway for monitoring purposes.
Network Server (Prepare HTTP)
Prepares the received data into an HTTP request format, ready for submission to ThingSpeak.
http request
Sends the data to ThingSpeak using an HTTP POST or GET request.
debug 1
Shows the server response from ThingSpeak, confirming if the upload was successful or if there were errors.

Flow Output and Results

The outputs below were recorded during the Node-RED simulation. They show the sensor data being created, sent through the simulated LoRa system, and uploaded to ThingSpeak. Debug nodes were used to show the data at each important step.

```
4/27/2025, 8:03:58 PM   node: debug 1
sensors/data : msg.payload : Object
▾object
  api_key: "KI4B82BPA6ICJR9S"
  field1: 26
  field2: 43
4/27/2025, 8:04:03 PM   node: debug 2
msg.payload : number
2025-04-27T18:04:03.155Z
4/27/2025, 8:04:03 PM   node: debug 3
sensors/data : msg.payload : Object
▾object
  temperature: 26
  humidity: 57
```





Timespeak Public Channel: https://thingspeak.mathworks.com/channels/2937282

LoRaSim Simulation Report

This report addresses **Exercise Question 3 (EQ3)** from the paper *"Do LoRa Low-Power Wide-A Networks Scale?"* by M. Bor et al.

The aim is to:

- Reproduce **Figure 5** and **Figure 7** using the **LoRaSim** simulator.
- Follow the simulation parameters as closely as possible.
- Present the results in a similar format for direct comparison.

In [1]:
```
! wget https://www.lancaster.ac.uk/scc/sites/lora/lorasim-20170710.tgz
! tar -xvf lorasim-20170710.tgz

!sudo apt-get install python2 && curl https://bootstrap.pypa.io/pip/2.7/g
!sudo apt install python-tk
!pip2 install -r 'lorasim/requirements.txt'
```

```
--2025-04-26 23:29:21--  https://www.lancaster.ac.uk/scc/sites/lora/
lorasim-20170710.tgz
Resolving www.lancaster.ac.uk (www.lancaster.ac.uk)... 148.88.65.80,
2001:630:80:2ff::80:1
Connecting to www.lancaster.ac.uk (www.lancaster.ac.uk)|148.88.65.80|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 112640 (110K) [application/x-gzip]
Saving to: ‘lorasim-20170710.tgz.1’

lorasim-20170710.tg 100%[===================>] 110.00K   320KB/s    in 0.3s

2025-04-26 23:29:22 (320 KB/s) - ‘lorasim-20170710.tgz.1’ saved [112640/11264

lorasim/
lorasim/loraDir.py
lorasim/loraDirMulBS.py
lorasim/oneDirectionalLoraIntf.py
lorasim/requirements.txt
lorasim/directionalLoraIntf.py
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python2 is already the newest version (2.7.18-3).
0 upgraded, 0 newly installed, 0 to remove and 34 not upgraded.
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curre
                                 Dload  Upload   Total   Spent    Left  Speed
100 1863k  100 1863k    0     0  6293k      0 --:--:-- --:--:-- --:--:-- 6316
Collecting pip<21.0
  Using cached pip-20.3.4-py2.py3-none-any.whl (1.5 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 20.3.4
    Uninstalling pip-20.3.4:
      Successfully uninstalled pip-20.3.4
Successfully installed pip-20.3.4
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python-tk is already the newest version (2.7.18-1build1).
0 upgraded, 0 newly installed, 0 to remove and 34 not upgraded.
Requirement already satisfied: numpy==1.11.1 in /usr/local/lib/python2.7/dist
packages (from -r lorasim/requirements.txt (line 1)) (1.11.1)
Requirement already satisfied: simpy==3.0.10 in /usr/local/lib/python2.7/dist
packages (from -r lorasim/requirements.txt (line 2)) (3.0.10)
Requirement already satisfied: matplotlib==2.0.0 in /usr/local/lib/python2.7/
packages (from -r lorasim/requirements.txt (line 3)) (2.0.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python2.7/dist-pac
(from matplotlib==2.0.0->-r lorasim/requirements.txt (line 3)) (1.17.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python2.7/di
packages (from matplotlib==2.0.0->-r lorasim/requirements.txt (line 3))
(2.9.0.post0)
Requirement already satisfied: pytz in /usr/local/lib/python2.7/dist-packages
matplotlib==2.0.0->-r lorasim/requirements.txt (line 3)) (2025.2)
Requirement already satisfied: functools32 in /usr/local/lib/python2.7/dist-
packages (from matplotlib==2.0.0->-r lorasim/requirements.txt (line 3))
(3.2.3.post2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python2.7/dist-
packages (from matplotlib==2.0.0->-r lorasim/requirements.txt (line 3)) (0.10
Requirement already satisfied: pyparsing!=2.0.0,!=2.0.4,!=2.1.2,!=2.1.6,>=1.5
```

In [4]:
```
import os
import pandas as pd
import math
import subprocess
import matplotlib
import matplotlib.pyplot as plt
```

In [5]:
```
def simulate(n_nodes, tx_rate, exp, duration):
    env = os.environ.copy()
    env["MPLBACKEND"] = "Agg"

    # Use subprocess.run to execute the command and capture output
    result = subprocess.run(
        [
            "python2",
            "lorasim/loraDir.py",
            str(int(n_nodes)),
            str(int(tx_rate)),
            str(int(exp)),
            str(int(duration)),
            "1"
        ],
        env=env,
        capture_output=True,
        text=True,  # Capture output as text
    )

    # Print the output
    # print(result.stdout)
    # print(result.stderr)
```

In [6]:
```
duration = 86400000
tx_rate = 1e6

for n_nodes in list(range(1,90,49)) + list(range(100,1000,100)) + list(rai
  print(f"Simulating {n_nodes} nodes")
  simulate(n_nodes, tx_rate, 4, duration)
  simulate(n_nodes, tx_rate, 3, duration)
  simulate(n_nodes, tx_rate, 5, duration)
```

```
Simulating 1 nodes
Simulating 50 nodes
Simulating 100 nodes
Simulating 200 nodes
Simulating 300 nodes
Simulating 400 nodes
Simulating 500 nodes
Simulating 600 nodes
Simulating 700 nodes
Simulating 800 nodes
Simulating 900 nodes
Simulating 1000 nodes
Simulating 1200 nodes
Simulating 1400 nodes
Simulating 1600 nodes
```

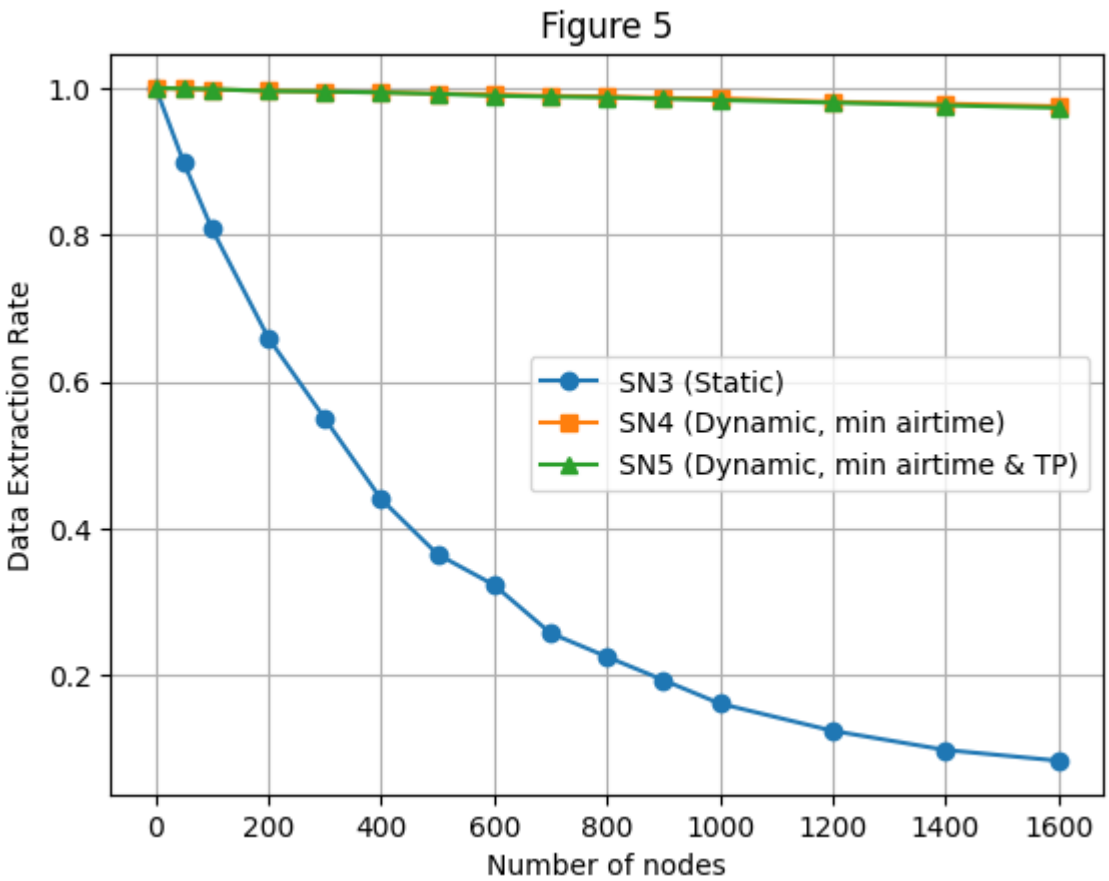
In [7]:
```
data3 = pd.read_csv("exp4.dat", sep=" ")
data4 = pd.read_csv("exp3.dat", sep=" ")
data5 = pd.read_csv("exp5.dat", sep=" ")

data3["der"] = (data3["nrTransmissions"] - data3["nrCollisions"]) / data3
data4["der"] = (data4["nrTransmissions"] - data4["nrCollisions"]) / data4
data5["der"] = (data5["nrTransmissions"] - data5["nrCollisions"]) / data5
```

Reproduction of Figure 5

In this section, we simulate the results corresponding to **Figure 5** from the paper. The parameter match those described in the original experimental setup.

In [8]:
```
plt.plot(data3["#nrNodes"], data3["der"], label="SN3 (Static)",marker='o'
plt.plot(data4["#nrNodes"], data4["der"], label="SN4 (Dynamic, min airtim
plt.plot(data5["#nrNodes"], data5["der"], label="SN5 (Dynamic, min airtim
plt.title("Figure 5")
plt.xlabel("Number of nodes")
plt.ylabel("Data Extraction Rate")
plt.legend()
plt.grid()
plt.show()
```



In [9]:
```
def simulate_multibs(n_nodes, tx_rate, exp, duration, sinks):
    env = os.environ.copy()
    env["MPLBACKEND"] = "Agg"

    result = subprocess.run(
        [
            "python2",
            "lorasim/loraDirMulBS.py",
            str(int(n_nodes)),
            str(int(tx_rate)),
            str(int(exp)),
            str(int(duration)),
            str(int(sinks)),
            "1"
        ],
        env=env,
        capture_output=True,
        text=True,
    )

    # Print the output
    # print(result.stdout)
    # print(result.stderr)
```

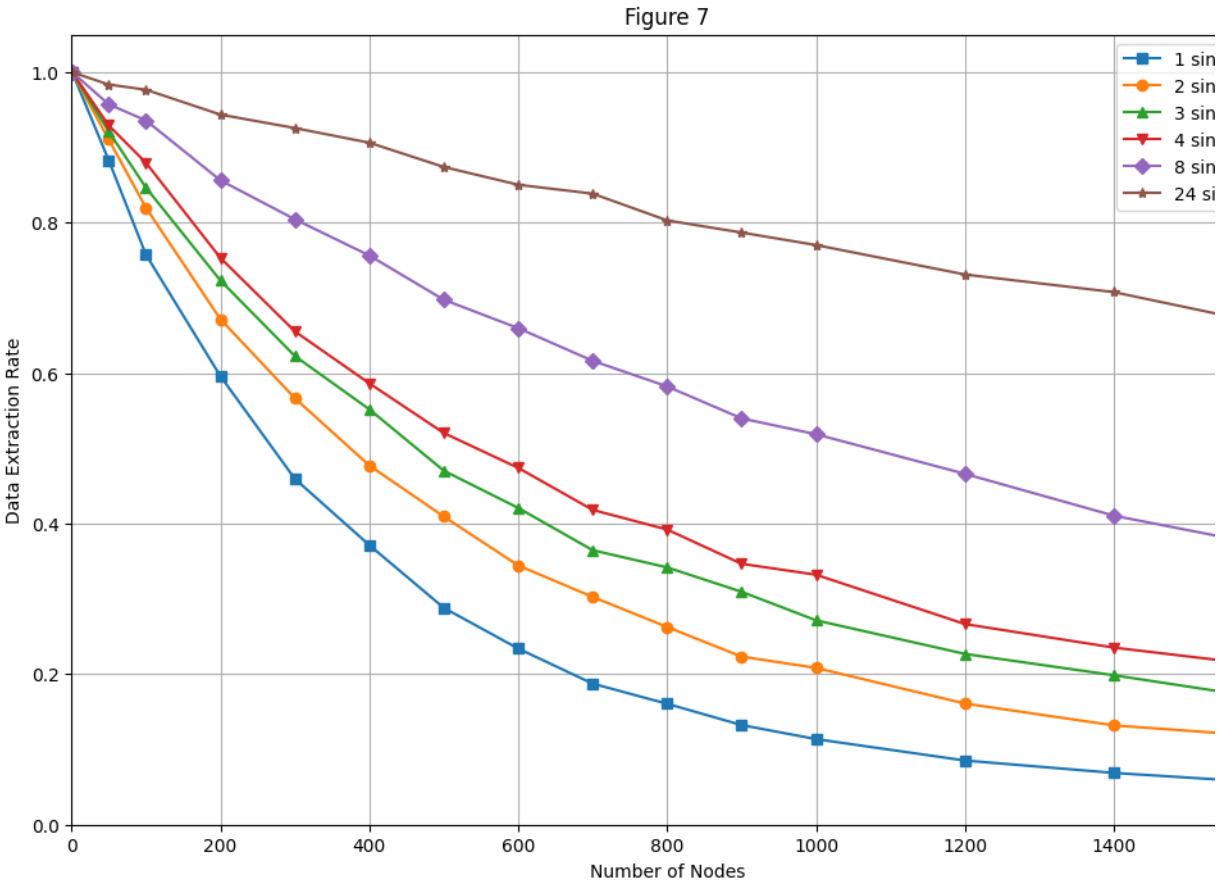
In [10]:
```
duration = 86400000  # 1 day in milliseconds
tx_rate = 1e6        # packet interval
sinks_list = [1, 2, 3, 4, 8, 24]
nodes_range = list(range(1,90,49)) + list(range(100,1000,100)) + list(ran

# Run simulations
for sinks in sinks_list:
    for n_nodes in nodes_range:
        print(f"Simulating {n_nodes} nodes for {sinks} sinks")
        simulate_multibs(n_nodes, tx_rate, 0, duration, sinks)
```

```
Simulating 1 nodes for 1 sinks
Simulating 50 nodes for 1 sinks
Simulating 100 nodes for 1 sinks
Simulating 200 nodes for 1 sinks
Simulating 300 nodes for 1 sinks
Simulating 400 nodes for 1 sinks
Simulating 500 nodes for 1 sinks
Simulating 600 nodes for 1 sinks
Simulating 700 nodes for 1 sinks
Simulating 800 nodes for 1 sinks
Simulating 900 nodes for 1 sinks
Simulating 1000 nodes for 1 sinks
Simulating 1200 nodes for 1 sinks
Simulating 1400 nodes for 1 sinks
Simulating 1600 nodes for 1 sinks
Simulating 1 nodes for 2 sinks
Simulating 50 nodes for 2 sinks
Simulating 100 nodes for 2 sinks
Simulating 200 nodes for 2 sinks
Simulating 300 nodes for 2 sinks
Simulating 400 nodes for 2 sinks
Simulating 500 nodes for 2 sinks
Simulating 600 nodes for 2 sinks
Simulating 700 nodes for 2 sinks
Simulating 800 nodes for 2 sinks
Simulating 900 nodes for 2 sinks
Simulating 1000 nodes for 2 sinks
Simulating 1200 nodes for 2 sinks
Simulating 1400 nodes for 2 sinks
Simulating 1600 nodes for 2 sinks
Simulating 1 nodes for 3 sinks
Simulating 50 nodes for 3 sinks
Simulating 100 nodes for 3 sinks
Simulating 200 nodes for 3 sinks
Simulating 300 nodes for 3 sinks
Simulating 400 nodes for 3 sinks
Simulating 500 nodes for 3 sinks
Simulating 600 nodes for 3 sinks
Simulating 700 nodes for 3 sinks
Simulating 800 nodes for 3 sinks
Simulating 900 nodes for 3 sinks
Simulating 1000 nodes for 3 sinks
Simulating 1200 nodes for 3 sinks
Simulating 1400 nodes for 3 sinks
Simulating 1600 nodes for 3 sinks
Simulating 1 nodes for 4 sinks
Simulating 50 nodes for 4 sinks
Simulating 100 nodes for 4 sinks
Simulating 200 nodes for 4 sinks
Simulating 300 nodes for 4 sinks
Simulating 400 nodes for 4 sinks
Simulating 500 nodes for 4 sinks
Simulating 600 nodes for 4 sinks
Simulating 700 nodes for 4 sinks
Simulating 800 nodes for 4 sinks
Simulating 900 nodes for 4 sinks
Simulating 1000 nodes for 4 sinks
Simulating 1200 nodes for 4 sinks
Simulating 1400 nodes for 4 sinks
Simulating 1600 nodes for 4 sinks
Simulating 1 nodes for 8 sinks
```

In [11]:
```
# Read all data
data = {}
for sinks in sinks_list:
    filename = f"exp0BS{sinks}.dat"
    d = pd.read_csv(filename, sep=" ", comment="#", header=None, names=["
    data[sinks] = d
```

Reproduction of Figure 7

Here we simulate and plot the results corresponding to **Figure 7** from the paper, maintaining alig
with the original parameters and conditions.

In [12]:
```
# Plotting
plt.figure(figsize=(12,8))
markers = {1: 's', 2: 'o', 3: '^', 4: 'v', 8: 'D', 24: '*'}
for sinks in sinks_list:
    plt.plot(data[sinks]["nodes"], data[sinks]["DER"], label=f"{sinks} si

plt.xlabel("Number of Nodes")
plt.ylabel("Data Extraction Rate")
plt.title("Figure 7")
plt.grid(True)
plt.legend()
plt.ylim(0, 1.05)
plt.xlim(0, 1600)
plt.show()
```